
Creer et gerer une organisation

via les endpoints kernel-core

100% API — aucune insertion en base — local **et** production

Ce guide permet a **n'importe qui** de creer son compte, devenir **OWNER**, creer et configurer une organisation (services, roles, permissions, membres) **uniquement** avec les API publiees par le **kernel-core**. Toutes les commandes sont des **curl** copiables.

Auteur : ALD • azangueleonel9@gmail.com

Cible	Local (localhost:8080) et Production (kernel-core.yowjob.com)
Pre-requis	curl, jq, une ClientApplication valide
Principe	tout passe par les endpoints /api/**; aucune ecriture SQL
Date	16 juin 2026 (rev. : verification email + approbation)

Table des matières

1	Prerequis et configuration	2
1.1	Variables d'environnement (local vs production)	2
1.2	Concepts	2
2	Etape 1 — Inscrire le premier utilisateur	2
3	Etape 2 — Se connecter (et activer la MFA si admin)	3
3.1	Login simple (compte sans MFA)	3
3.2	Login en 2 etapes (compte avec MFA activee)	3
3.3	Activer la MFA via l'API (comptes admin — sans toucher la base)	3
3.4	Verifier l'adresse email (envoi SMTP reel)	4
4	Etape 3 — Devenir OWNER	5
4.1	Option A — Rejoindre une organisation existante	5
4.2	Option B — Se faire attribuer OWNER par un admin du tenant	5
5	Etape 4 — Creer le business actor et l'organisation	6
5.1	Business actor (proprietaire)	6
5.2	Creer l'organisation	6
5.3	Voir mes organisations	6
5.4	Approbation et gouvernance de l'organisation	6
6	Etape 5 — Souscrire l'organisation aux services	7
7	Etape 6 — Roles et permissions	7
7.1	Lister les permissions disponibles	7
7.2	Lister les roles de l'organisation / du tenant	7
7.3	(Optionnel) Provisionner les roles par default	7
7.4	Creer un role custom	7
7.5	Affecter un role a un utilisateur	8
7.6	Lister les roles d'un utilisateur	8
8	Etape 7 — Gerer les membres (employes)	8
8.1	Inviter un employe (avec un role, ou des permissions directes)	8
8.2	Lister les employes	9
9	Reference des endpoints	9
10	Erreurs frequentes	10

1. Prerequis et configuration

1.1 Variables d'environnement (local vs production)

```
# ---- PRODUCTION ----
export BASE_URL="https://kernel-core.yowyob.com"
export CLIENT_ID="<client-id-prod>"      # fourni par le DevOps (NE PAS commiter)
export API_KEY="<api-key-prod>"         # secret
export TENANT_ID="<uuid-tenant-prod>"

# ---- LOCAL ----
# export BASE_URL="http://localhost:8080"
# export CLIENT_ID="dev-platform-backend"
# export API_KEY="dev-api-key"
# export TENANT_ID="11111111-1111-1111-1111-111111111111"
```

Regles communes a TOUS les appels

- Tout `/api/**` exige le couple `X-Client-Id` + `X-API-Key` d'une `ClientApplication` valide. En prod, les identifiants `dev-*` sont **rejetés** (401).
- Les endpoints proteges exigent en plus `Authorization: Bearer <token>`.
- Les operations liees a une organisation exigent l'en-tete `X-Organization-Id`.
- Le `accessToken` expire vite (`expiresInSeconds` = 900, soit 15 min). Un 403 / `Access Denied` en cours de manip signifie souvent un token perime : reconnectez-vous.
- **Mot de passe** : ≥ 10 caracteres avec majuscule, minuscule, chiffre **et** symbole.
- `accountType` : utiliser `BUSINESS` (`PROSPECT` declenche une erreur d'onboarding).

1.2 Concepts

Modele de donnees

- **Tenant** : espace d'isolation racine (un UUID). Tous les objets en dependent (en-tete `X-Tenant-Id` au login/sign-up).
- **ClientApplication** : l'application appelante (`X-Client-Id` / `X-API-Key`).
- **User** : un compte (login). **BusinessActor** : l'entite metier (proprietaire, employe...) rattachée a un user.
- **Organisation** : creee par un **OWNER** ; on lui souscrit des **services** (`ACCOUNTING`, `COMMERCIAL`, `CASHIER`...).
- **Role** = ensemble de **permissions** (ex. `accounting:write`). **Scope** : `TENANT` (vaut pour tout le tenant) ou `ORGANISATION` (limite a une org). Les permissions resolues portent un suffixe `#TENANT` selon le scope.

2. Etape 1 — Inscrire le premier utilisateur

`POST /api/auth/sign-up` — avec `tenantId` explicite (premier compte d'un tenant). Renvoie directement un `accessToken` si le compte n'a pas de MFA.

```
export SIGNUP=$(curl -s -X POST "$BASE_URL/api/auth/sign-up" \
  -H "X-Client-Id: $CLIENT_ID" -H "X-API-Key: $API_KEY" \
  -H "Content-Type: application/json" \
  -d '{
    "tenantId": "'"$TENANT_ID"'",
```

```

    "firstName": "AZANGUE", "lastName": "DELMAT",
    "username": "azangue.delmatt", "email": "owner@example.com",
    "phoneNumber": "+237690112233",
    "password": "Secret123!", "accountType": "BUSINESS", "businessType": "RETAIL"
  })
export ACCESS_TOKEN=$(echo "$SIGNUP" | jq -r '.data.accessToken')
export USER_ID=$(echo "$SIGNUP" | jq -r '.data.id')
export AUTH_ACTOR_ID=$(echo "$SIGNUP" | jq -r '.data.actorId')
echo "$SIGNUP" | jq '{userId:.data.id, actorId:.data.actorId, hasToken:(.data.accessToken!=null)}'
}

```

Le sign-up avec tenantId n'attribue PAS le role OWNER

Pour creer une organisation (organizations:write), il faut d'abord devenir OWNER : voir l'etape 3.

3. Etape 2 — Se connecter (et activer la MFA si admin)

3.1 Login simple (compte sans MFA)

```

export ACCESS_TOKEN=$(curl -s -X POST "$BASE_URL/api/auth/login" \
  -H "X-Client-Id: $CLIENT_ID" -H "X-API-Key: $API_KEY" \
  -H "X-Tenant-Id: $TENANT_ID" -H "Content-Type: application/json" \
  -d '{"principal":"owner@example.com","password":"Secret123!"}' \
  | jq -r '.data.accessToken')

```

3.2 Login en 2 etapes (compte avec MFA activee)

Si la reponse du login est HTTP 202 avec data.nextStep=CONFIRM_MFA :

```

export LOGIN=$(curl -s -X POST "$BASE_URL/api/auth/login" \
  -H "X-Client-Id: $CLIENT_ID" -H "X-API-Key: $API_KEY" \
  -H "X-Tenant-Id: $TENANT_ID" -H "Content-Type: application/json" \
  -d '{"principal":"owner@example.com","password":"Secret123!"}')
export MFA_TOKEN=$(echo "$LOGIN" | jq -r '.data.mfaToken')
export OTP=$(echo "$LOGIN" | jq -r '.data.codePreview') # LOCAL ; en PROD : lire l'email/SMS

export ACCESS_TOKEN=$(curl -s -X POST "$BASE_URL/api/auth/login/mfa/confirm" \
  -H "X-Client-Id: $CLIENT_ID" -H "X-API-Key: $API_KEY" \
  -H "X-Tenant-Id: $TENANT_ID" -H "Content-Type: application/json" \
  -d '{"mfaToken":"'"$MFA_TOKEN"'", "code":"'"$OTP"'"}' | jq -r '.data.accessToken')

```

3.3 Activer la MFA via l'API (comptes admin — sans toucher la base)

Les comptes admin privileges doivent avoir la MFA activee en permanence (sinon 403 MFA_REQUIRED_FOR_ADMIN). Tout se fait par API :

```

# 1) Demander un challenge (canal EMAIL/SMS)
export CH=$(curl -s -X POST "$BASE_URL/api/auth/mfa/enable" \
  -H "X-Client-Id: $CLIENT_ID" -H "X-API-Key: $API_KEY" \
  -H "Authorization: Bearer $ACCESS_TOKEN" -H "Content-Type: application/json" \
  -d '{"channel":"EMAIL"}')
export CH_TOKEN=$(echo "$CH" | jq -r '.data.token')
export CH_CODE=$(echo "$CH" | jq -r '.data.codePreview') # LOCAL ; PROD : email/SMS

# 2) Confirmer
curl -s -X POST "$BASE_URL/api/auth/mfa/confirm" \
  -H "X-Client-Id: $CLIENT_ID" -H "X-API-Key: $API_KEY" \

```

```
-H "Authorization: Bearer $ACCESS_TOKEN" -H "Content-Type: application/json" \
-d '{"challengeToken":"'"$CH_TOKEN"'", "code":"'"$CH_CODE"'"}' | jq '{mfa:.data.mfaEnabled}'
```

Decoder son token

```
echo "$ACCESS_TOKEN" | cut -d. -f2 | tr '._' '/' | base64 -d 2>/dev/null \
| jq '{sub,tid,oid,permissions}'
```

3.4 Verifier l'adresse email (envoi SMTP reel)

A la creation, `emailVerified=false`. La verification se fait en deux temps : l'emission d'un challenge (qui **declenche un envoi SMTP**) puis la confirmation du token reçu par email.

```
# 1) Declencher l'envoi de l'email de verification (utilisateur courant)
curl -s -X POST "$BASE_URL/api/auth/email-verification/request" \
-H "X-Client-Id: $CLIENT_ID" -H "X-API-Key: $API_KEY" \
-H "X-Tenant-Id: $TENANT_ID" -H "Authorization: Bearer $ACCESS_TOKEN" \
| jq '{mode:.data.deliveryMode, expiresIn:.data.expiresInSeconds}'
# deliveryMode == "SMTP" -> email reellement envoye
# deliveryMode == "PREVIEW_ONLY" -> email desactive, token expose dans la reponse

# 2) Confirmer avec le token reçu dans l'email (parametre ?token=... du lien)
export VERIF_TOKEN="<token reçu par email>"
curl -s -X POST "$BASE_URL/api/auth/email-verification/confirm" \
-H "X-Client-Id: $CLIENT_ID" -H "X-API-Key: $API_KEY" \
-H "X-Tenant-Id: $TENANT_ID" -H "Authorization: Bearer $ACCESS_TOKEN" \
-H "Content-Type: application/json" \
-d '{"verificationToken":"'"$VERIF_TOKEN"'"}' | jq '{emailVerified:.data.emailVerified}'
```

Configuration SMTP (cote serveur)

L'envoi reel exige une config SMTP cote kernel (SPRING_MAIL_* + IWM_AUTH_EMAIL_ENABLED=true). Avec Gmail : SPRING_MAIL_HOST=smtp.gmail.com, PORT=587, USERNAME=<compte>, PASSWORD=<App Password 16 car.> — la 2FA doit etre active sur le compte Google (un mot de passe normal est refuse). En mode SMTP, `challengeTokenPreview` vaut null : le token n'est diffuse QUE par email (JWT typ=auth-email-verification, **non stocke en base**).

535 BadCredentials et faux positifs

Un 500 dont le log montre 535-5.7.8 Username and Password not accepted = App Password invalide/revoque. **Piege** : un request qui renvoie 200 ne prouve PAS l'envoi — en PREVIEW_ONLY il n'y a aucun appel SMTP. La preuve d'un envoi reel = `deliveryMode:"SMTP"` et absence de 535 dans les logs (ideal : reception effective de l'email).

Lien clique = page Whitelabel 404 ?

Le lien de l'email pointe vers IWM_AUTH_EMAIL_PUBLIC_BASE_URL + /auth/verify-email?token=.... Si cette base vise le kernel (:8080), la page n'existe pas (404) : c'est une page du **frontend** (:3000). L'email reste verifiable via l'API /email-verification/confirm ci-dessus. Pour des liens cliquables, pointer la base vers le frontend.

Membres crees via /api/auth/register : actorId requis

sign-up cree automatiquement un **business actor**; register (cote admin) ne le fait PAS. Sans actorId, email-verification/request echoue en NullPointerException (le token de verif embarque actorId). Creer d'abord le business actor et le rattacher au compte (etape 4).

4. Etape 3 — Devenir OWNER

Necessaire pour creer une organisation. Deux cas.

4.1 Option A — Rejoindre une organisation existante

```
# 1) Decouvrir les contextes d'inscription pour un code d'organisation
curl -s -X POST "$BASE_URL/api/auth/discover-sign-up-contexts" \
-H "X-Client-Id: $CLIENT_ID" -H "X-API-Key: $API_KEY" \
-H "Content-Type: application/json" \
-d '{"organizationCode":"ORG-KSM"}' | jq

# 2) S'inscrire avec le token retourne (le tenant est resolu cote serveur)
export SIGNUP_SELECTION_TOKEN=<data.selectionToken>
export CONTEXT_ID=<data.contexts[0].contextId>
curl -s -X POST "$BASE_URL/api/auth/sign-up" \
-H "X-Client-Id: $CLIENT_ID" -H "X-API-Key: $API_KEY" \
-H "Content-Type: application/json" \
-d '{
  "signUpSelectionToken":"'${SIGNUP_SELECTION_TOKEN}'",
  "contextId":"'${CONTEXT_ID}'",
  "firstName":"Marie","lastName":"Comptable","username":"mcomptable",
  "email":"membre@example.com","password":"Secret123!","accountType":"BUSINESS"
}' | jq
```

4.2 Option B — Se faire attribuer OWNER par un admin du tenant

Un compte admin du **memme tenant** (ex. `platform-admin`) attribue le role OWNER, via l'API d'administration :

```
# Lister les roles du tenant pour recuperer l'id du role OWNER
curl -s -X GET "$BASE_URL/api/administration/roles" \
-H "X-Client-Id: $CLIENT_ID" -H "X-API-Key: $API_KEY" \
-H "Authorization: Bearer $PLATFORM_ADMIN_TOKEN" \
| jq '.data[]? // .[] | {code,id}'
export OWNER_ROLE_ID=<id du role OWNER>

# Attribuer OWNER (scope TENANT) a l'utilisateur cible
curl -s -X POST "$BASE_URL/api/administration/users/$USER_ID/roles" \
-H "X-Client-Id: $CLIENT_ID" -H "X-API-Key: $API_KEY" \
-H "Authorization: Bearer $PLATFORM_ADMIN_TOKEN" -H "Content-Type: application/json" \
-d '{"roleId":"'${OWNER_ROLE_ID}'","scopeType":"TENANT","scope":"TENANT"}' | jq
```

Reconnexion

Après toute modification de roles, **reconnectez-vous** (etape 2) : le kernel re-resout les permissions a chaque requete, mais votre *token* courant ne refletera les nouvelles permissions qu'après un nouveau login.

5. Etape 4 — Créer le business actor et l'organisation

5.1 Business actor (proprietaire)

```
export ACTOR=$(curl -s -X POST "$BASE_URL/api/actors/onboarding" \
-H "X-Client-Id: $CLIENT_ID" -H "X-API-Key: $API_KEY" \
-H "Authorization: Bearer $ACCESS_TOKEN" -H "Content-Type: application/json" \
-d '{"name":"KSM Owner","businessId":"BU-001","role":"OWNER","type":"BUSINESS",
      "isIndividual":true,"isActive":true}')
export BUSINESS_ACTOR_ID=$(echo "$ACTOR" | jq -r '.data.id')
```

5.2 Créer l'organisation

```
export ORG=$(curl -s -X POST "$BASE_URL/api/organizations" \
-H "X-Client-Id: $CLIENT_ID" -H "X-API-Key: $API_KEY" \
-H "Authorization: Bearer $ACCESS_TOKEN" -H "Content-Type: application/json" \
-d '{"businessActorId":"'"$BUSINESS_ACTOR_ID"'',"code":"ORG-KSM",
      "legalName":"KSM-ERP-ACCOUNTING","displayName":"KSM SARL",
      "organizationType":"PRIVATE_COMPANY"}')
export ORGANIZATION_ID=$(echo "$ORG" | jq -r '.data.id')
echo "ORGANIZATION_ID=$ORGANIZATION_ID"
```

5.3 Voir mes organisations

```
curl -s -X GET "$BASE_URL/api/organizations/my" \
-H "X-Client-Id: $CLIENT_ID" -H "X-API-Key: $API_KEY" \
-H "Authorization: Bearer $ACCESS_TOKEN" | jq
```

403 sur /api/organizations

Signifie que le compte n'a pas `organizations:write` (= pas OWNER) : revenez a l'etape 3.

5.4 Approbation et gouvernance de l'organisation

A la creation, l'organisation naît en `governanceStatus = PENDING_APPROVAL` (et `status = PENDING_APPROVAL`) elle existe mais n'est pas validee. Un acteur **gouverneur** l'approuve :

```
curl -s -X POST "$BASE_URL/api/organizations/$ORGANIZATION_ID/approve" \
-H "X-Client-Id: $CLIENT_ID" -H "X-API-Key: $API_KEY" \
-H "X-Tenant-Id: $TENANT_ID" -H "Authorization: Bearer $ACCESS_TOKEN" \
-H "Content-Type: application/json" \
-d '{"reason":"Validation owner"}' \
| jq '{governance:.data.governanceStatus, status:.data.status, by:.data.governedByUserId}'
# -> governanceStatus:"APPROVED", status:"APPROVED"
```

Qui peut approuver ? (canGovernOrganizations)

Il faut un **contexte utilisateur** + l'une des permissions : `administration:govern:organizations`, `system:admin`, `iam:admin` ou `tenant:admin`. Un OWNER possède `tenant:admin` sur son tenant : il peut donc **approuver sa propre organisation**. L'action est transactionnelle et auditee (`ORGANIZATION_APPROVED`) ; elle renseigne `governedByUserId` et `governedAt`.

Actions de gouvernance associees (meme garde `canGovernOrganizations`, corps optionnel `{"reason":"..."}`) :

- `POST /api/organizations/{org}/reject` — refuser le dossier (REJECTED).
- `POST /api/organizations/{org}/reopen` — rouvrir un dossier traite.
- `POST /api/organizations/{org}/suspend` — suspendre (cascade aux agences).

6. Etape 5 — Souscrire l'organisation aux services

Chaque module est filtré par service. Souscrivez ceux dont vous avez besoin (ACCOUNTING, COMMERCIAL, BILLING, CASHIER, HRM...):

```
for SVC in ACCOUNTING COMMERCIAL BILLING CASHIER; do
  curl -s -X POST "$BASE_URL/api/organizations/$ORGANIZATION_ID/services" \
    -H "X-Client-Id: $CLIENT_ID" -H "X-API-Key: $API_KEY" \
    -H "X-Organization-Id: $ORGANIZATION_ID" \
    -H "Authorization: Bearer $ACCESS_TOKEN" -H "Content-Type: application/json" \
    -d '{"serviceCode": "'"$SVC"'", "requestQuotaLimit": 10000,
      "requestQuotaWindowSeconds": 3600}' \
    | jq -rc '{svc: "'"$SVC"'", ok: .success, err: .errorCode}'
done
```

7. Etape 6 — Roles et permissions

7.1 Lister les permissions disponibles

```
curl -s -X GET "$BASE_URL/api/administration/permissions" \
  -H "X-Client-Id: $CLIENT_ID" -H "X-API-Key: $API_KEY" \
  -H "X-Organization-Id: $ORGANIZATION_ID" \
  -H "Authorization: Bearer $ACCESS_TOKEN" | jq
```

Exemples courants : `organizations:write`, `administration:read/write`, `administration:roles:read/write`, `administration:assignments:write`, `accounting:read/write/post`, `third-parties:read/write`, `treasury:read/write`.

7.2 Lister les roles de l'organisation / du tenant

```
curl -s -X GET "$BASE_URL/api/administration/roles" \
  -H "X-Client-Id: $CLIENT_ID" -H "X-API-Key: $API_KEY" \
  -H "X-Organization-Id: $ORGANIZATION_ID" \
  -H "Authorization: Bearer $ACCESS_TOKEN" | jq
```

7.3 (Optionnel) Provisionner les roles par défaut

```
curl -s -X POST "$BASE_URL/api/administration/roles/defaults" \
  -H "X-Client-Id: $CLIENT_ID" -H "X-API-Key: $API_KEY" \
  -H "X-Organization-Id: $ORGANIZATION_ID" \
  -H "Authorization: Bearer $ACCESS_TOKEN" | jq
```

Codes reserves

Les codes `ACCOUNTANT`, `ORGANIZATION_ADMIN`, `HR_MANAGER`... sont **reserves** (erreur `ADMIN_ROLE_PROTECTED`). Utilisez d'abord les templates par défaut, ou choisissez un code libre.

7.4 Créer un role custom


```
# Role administrateur d'organisation
export ORG_ADMIN_ROLE_ID=$(curl -s -X POST "$BASE_URL/api/administration/roles" \
  -H "X-Client-Id: $CLIENT_ID" -H "X-API-Key: $API_KEY" \
  -H "X-Organization-Id: $ORGANIZATION_ID" \
  -H "Authorization: Bearer $ACCESS_TOKEN" -H "Content-Type: application/json" \
  -d '{"code": "ORG_ADMIN_CUSTOM", "name": "Administrateur Organisation",
    "scopeType": "ORGANIZATION",
    "permissions": ["organizations:write", "administration:read", "administration:write",
      "administration:roles:read", "administration:roles:write",
      "administration:assignments:write"]}') | jq -r '.data.id')

# Role comptable
export ACCOUNTANT_ROLE_ID=$(curl -s -X POST "$BASE_URL/api/administration/roles" \
  -H "X-Client-Id: $CLIENT_ID" -H "X-API-Key: $API_KEY" \
  -H "X-Organization-Id: $ORGANIZATION_ID" \
  -H "Authorization: Bearer $ACCESS_TOKEN" -H "Content-Type: application/json" \
  -d '{"code": "COMPTA_CUSTOM", "name": "Comptable", "scopeType": "ORGANIZATION",
    "permissions": ["accounting:read", "accounting:write", "accounting:post"]}') | jq -r '.data.id')
```

ADMIN_PERMISSION_INVALID

Une permission de scope supérieur à celui du rôle est refusée (Permission ... exceeds the role scope ORGANIZATION). Alignez `scopeType` et les permissions, ou utilisez `scopeType: "TENANT"`.

7.5 Affecter un rôle à un utilisateur

```
export TARGET_USER_ID="<userId cible>"
# Scope organisation
curl -s -X POST "$BASE_URL/api/administration/users/$TARGET_USER_ID/roles" \
  -H "X-Client-Id: $CLIENT_ID" -H "X-API-Key: $API_KEY" \
  -H "X-Organization-Id: $ORGANIZATION_ID" \
  -H "Authorization: Bearer $ACCESS_TOKEN" -H "Content-Type: application/json" \
  -d '{"roleId": "'$ACCOUNTANT_ROLE_ID'", "scopeType": "ORGANIZATION",
    "scopeId": "'$ORGANIZATION_ID'", "scope": "ORGANIZATION"}' | jq
```

7.6 Lister les rôles d'un utilisateur

```
curl -s -X GET "$BASE_URL/api/administration/users/$TARGET_USER_ID/roles" \
  -H "X-Client-Id: $CLIENT_ID" -H "X-API-Key: $API_KEY" \
  -H "X-Organization-Id: $ORGANIZATION_ID" \
  -H "Authorization: Bearer $ACCESS_TOKEN" | jq
```

8. Etape 7 — Gérer les membres (employés)

8.1 Inviter un employé (avec un rôle, ou des permissions directes)

```
# Avec un roleId
curl -s -X POST "$BASE_URL/api/employees/invite?organizationId=$ORGANIZATION_ID" \
  -H "X-Client-Id: $CLIENT_ID" -H "X-API-Key: $API_KEY" \
  -H "X-Organization-Id: $ORGANIZATION_ID" \
  -H "Authorization: Bearer $ACCESS_TOKEN" -H "Content-Type: application/json" \
  -d '{"email": "comptable@example.com", "roleId": "'$ACCOUNTANT_ROLE_ID'"}' | jq

# Avec des permissions directes
```

```
curl -s -X POST "$BASE_URL/api/employees/invite?organizationId=$ORGANIZATION_ID" \
-H "X-Client-Id: $CLIENT_ID" -H "X-API-Key: $API_KEY" \
-H "X-Organization-Id: $ORGANIZATION_ID" \
-H "Authorization: Bearer $ACCESS_TOKEN" -H "Content-Type: application/json" \
-d '{"email": "caissier@example.com",
  "permissions": ["accounting:read", "cashier:read", "cashier:write"]}' | jq
```

Pre-requis service

L'invitation d'employés requiert le service HRM souscrit à l'organisation (étape 5).

8.2 Lister les employés

```
curl -s -X GET "$BASE_URL/api/employees?organizationId=$ORGANIZATION_ID" \
-H "X-Client-Id: $CLIENT_ID" -H "X-API-Key: $API_KEY" \
-H "X-Organization-Id: $ORGANIZATION_ID" \
-H "Authorization: Bearer $ACCESS_TOKEN" | jq
```

9. Reference des endpoints

Endpoint	Methode	Role
/api/auth/sign-up	POST	inscription
/api/auth/discover-sign-up-contexts	POST	rejoindre une org
/api/auth/login	POST	connexion (étape 1)
/api/auth/login/mfa/confirm	POST	connexion (étape 2 MFA)
/api/auth/mfa/enable	POST	activer la MFA
/api/auth/mfa/confirm		
/api/auth/email-verification/request	POST	envoyer l'email de verif (SMTP)
/api/auth/email-verification/confirm	POST	confirmer l'email (token reçu)
/api/auth/register	POST	créer un user dans un tenant existant (admin)
/api/actors/onboarding	POST	créer le business actor
/api/organizations	POST	créer une organisation
/api/organizations/{org}/approve	POST	approuver l'organisation
/api/organizations/{org}/reject reopen suspend	POST	gouvernance org
/api/organizations/my	GET	mes organisations
/api/organizations/{org}/services	POST	souscrire un service
/api/administration/permissions	GET	lister les permissions
/api/administration/roles	GET / POST	lister / créer un rôle
/api/administration/roles/defaults	POST	rôles par défaut
/api/administration/users/{id}/roles	GET / POST	rôles d'un user / affecter
/api/employees/invite?organizationId=	POST	inviter un employé

Endpoint	Methode	Role
/api/employees?organizationId=	GET	lister les employes

10. Erreurs frequentes

Erreur	Cause	Resolution
401 sur /api/**	X-Client-Id / X-API-Key invalides (dev-* en prod)	utiliser la ClientApplication de prod
tenantId or signUpSelectionToken/contextId is required	signUpSelectionToken/contextId is required	fournir tenantId (ou le couple token+contextId)
500 au sign-up	accountType:"PROSPECT"	utiliser "BUSINESS"
400 AUTH_INVALID_REQUEST	mot de passe trop faible	≥ 10 car. (Maj+min+chiffre+symbole)
login 202 CONFIRM_MFA	MFA active	confirmer via /login/mfa/confirm
500 + 535-5.7.8 ... not accepted	App Password SMTP invalide/revoque	regenerer un App Password (2FA active)
request=200 mais pas d'email	mode PREVIEW_ONLY (SMTP off)	verifier deliveryMode:"SMTP" + config SPRING_MAIL_*
500 NullPointerException actorId	user register sans business actor	creer l'actor avant la verif email
org bloquee PENDING_APPROVAL	non approuvee	POST .../approve (role tenant:admin)
403 MFA_REQUIRED_FOR_ADMIN	admin sans MFA permanente	activer la MFA (/mfa/enable + /confirm)
403 sur /api/organizations	pas organizations:write (pas OWNER)	etape 3
ORGANIZATION_CONTEXT_REQUIRED	Den-tete X-Organization-Id absent	l'ajouter
ADMIN_ROLE_PROTECTED	code de role reserve	code libre / templates par default
ADMIN_PERMISSION_INVALID	permission hors scope du role	aligner scopeType / permissions
Access Denied en cours de manip	token perime (15 min)	se reconnecter

KSM-ERP / Yowyob — Auteur : ALD (azangueleonel9@gmail.com) — 16 juin 2026